

UNIT 2 – CORE AI CONCEPTS

Introduction to Machine Learning

Machine Learning (ML) is a branch of Artificial Intelligence that enables computers to learn patterns from data and make predictions or decisions without being explicitly programmed for every situation.

In traditional programming, a programmer provides rules to the computer. In machine learning, the computer learns useful rules or patterns from the given data.

Example

Suppose we want to identify whether an email is **Spam** or **Not Spam**. A machine-learning model can learn from previously labelled emails and then classify new emails.

Major Types of Machine Learning

Machine learning is mainly divided into:

1. **Supervised Learning**
2. **Unsupervised Learning**
3. **Reinforcement Learning**

For the syllabus topic we will cover:

- **KNN** → Supervised Learning
- **Naïve Bayes** → Supervised Learning
- **K-Means** → Unsupervised Learning

Classification

Classification is a supervised machine-learning technique in which a model predicts the **category or class label** of given input data.

The model is trained using labelled data.

Examples

- Email → Spam / Not Spam
- Student → Pass / Fail
- Tumor → Benign / Malignant
- Transaction → Fraud / Genuine
- Image → Cat / Dog

If the output contains two classes, it is called **Binary Classification**.

Example: Pass or Fail.

If there are more than two classes, it is called **Multi-class Classification**.

Example: Classifying an animal as Cat, Dog, Horse or Cow.

K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a supervised machine-learning algorithm used mainly for classification. It classifies a new data point according to the classes of its **K nearest data points** in the training dataset.

In simple words:

KNN assigns a class to a new data point by observing the classes of its nearest neighbors.

Here, **K represents the number of nearest neighbors considered for prediction.**

Basic Idea of KNN

Suppose a new unknown data point has to be classified as either **Class A** or **Class B**.

If we select:

K = 5

and among the five nearest neighbors:

- 4 belong to Class A
- 1 belongs to Class B

then the new data point will be classified as:

Class A: because Class A has the majority among the nearest neighbors.

The KNN algorithm works in the following steps:

Step 1: Select the value of **K**.

Step 2: Calculate the distance between the new data point and all training data points.

Step 3: Select the **K nearest data points** having the smallest distances.

Step 4: Check the class labels of these K nearest neighbors.

Step 5: Count the number of neighbors belonging to each class.

Step 6: Assign the class having the **majority vote** to the new data point.

Flow: New Data Point → Calculate Distance → Select K Nearest Neighbors → Majority Voting → Predicted Class

Distance Calculation in KNN

KNN requires a method to determine which points are nearest.

One of the most commonly used methods is **Euclidean Distance**.

For two points:

$$P = (x_1, y_1)$$

and

$$Q = (x_2, y_2)$$

the Euclidean distance is:

$$d(P, Q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

For n-dimensional data:

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Simple Example

Consider:

$$A = (2, 3)$$

and

$$B = (5, 7)$$

Then:

$$\begin{aligned} d &= \sqrt{(5 - 2)^2 + (7 - 3)^2} \\ &= \sqrt{3^2 + 4^2} \\ &= \sqrt{9 + 16} \\ &= \sqrt{25} = 5 \end{aligned}$$

Therefore, the Euclidean distance between A and B is 5.

How to Choose the Value of K?

Choosing K is important for the performance of the KNN algorithm.

Small value of K

If K is very small, the algorithm may become sensitive to noise and individual data points.

For example:

K = 1

means only the nearest data point determines the result.

Large value of K

A very large K may include points that are far away from the query point and may reduce classification accuracy.

For binary classification, an **odd value of K**, such as 3, 5 or 7, is often useful because it can reduce the possibility of a voting tie.

In practical machine-learning systems, the best value of K is usually selected using validation techniques.

Advantages of KNN

1. KNN is simple and easy to understand.
2. It is easy to implement.
3. It does not require a complex training process.
4. It can be used for both classification and regression.
5. It can work effectively for small datasets.
6. It can handle non-linear decision boundaries.

Disadvantages of KNN

1. Prediction can be slow for large datasets.
2. It requires calculation of distance from many training points.
3. Its performance depends on the value of K.
4. It is sensitive to irrelevant features.
5. Features with different numerical scales can affect distance calculations.
6. It requires relatively high memory because training data must be stored.

Applications of KNN

KNN can be used in:

- Pattern recognition
- Image classification
- Recommendation systems
- Medical diagnosis
- Handwriting recognition
- Credit-risk analysis

KNN Example – Movie Classification

Similar to the movie-based approach in the referenced KNN video, suppose movies are classified into two categories:

- **Action**
- **Romantic**

We use two features:

- Number of **Action Scenes**
- Number of **Romantic Scenes**

The following values are intentionally different from the video.

Movie	Action Scenes (X)	Romantic Scenes (Y)	Category
M1	8	2	Action
M2	7	3	Action
M3	9	4	Action
M4	2	8	Romantic
M5	3	7	Romantic
M6	4	9	Romantic

Now suppose a new movie **M7** contains:

$(6, 5)$

That means:

Action Scenes = 6

Romantic Scenes = 5

We want to classify M7 using:

$K = 3$

Step 1: Calculate Euclidean Distance

Euclidean distance is:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Distance from M1 (8,2)

$$\begin{aligned}d &= \sqrt{(8 - 6)^2 + (2 - 5)^2} \\&= \sqrt{4 + 9} = \sqrt{13} = 3.61\end{aligned}$$

Distance from M2 (7,3)

$$\begin{aligned}d &= \sqrt{(7 - 6)^2 + (3 - 5)^2} \\&= \sqrt{1 + 4} = \sqrt{5} = 2.24\end{aligned}$$

Distance from M3 (9,4)

$$\begin{aligned}d &= \sqrt{(9 - 6)^2 + (4 - 5)^2} \\&= \sqrt{9 + 1} = \sqrt{10} = 3.16\end{aligned}$$

Distance from M4 (2,8)

$$\begin{aligned}d &= \sqrt{(2 - 6)^2 + (8 - 5)^2} \\&= \sqrt{16 + 9} = \sqrt{25} = 5\end{aligned}$$

Distance from M5 (3,7)

$$\begin{aligned}d &= \sqrt{(3 - 6)^2 + (7 - 5)^2} \\&= \sqrt{9 + 4} = \sqrt{13} = 3.61\end{aligned}$$

Distance from M6 (4,9)

$$\begin{aligned}d &= \sqrt{(4 - 6)^2 + (9 - 5)^2} \\&= \sqrt{4 + 16} = \sqrt{20} = 4.47\end{aligned}$$

Step 2: Arrange the Distances

Movie	Distance	Category
M2	2.24	Action
M3	3.16	Action
M1	3.61	Action
M5	3.61	Romantic
M6	4.47	Romantic
M4	5.00	Romantic

There is a tie in distance at 3.61 between M1 and M5, so this particular dataset is not ideal for a deterministic $K=3$ classroom example unless a tie-breaking rule is specified. In an exam, it is better to avoid such ambiguity.

So, use the following slightly adjusted new movie instead:

$$M7 = (6, 4)$$

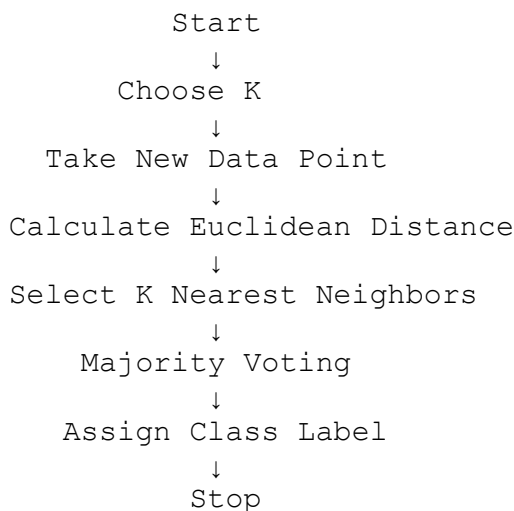
Its nearest neighbors are clearly dominated by the Action movies.

For $K = 3$, the three nearest neighbors are from the **Action** class.

Therefore:

M7 is classified as an Action Movie

KNN Flowchart



Naïve Bayes Classifier

Naïve Bayes is a supervised machine-learning classification algorithm based on Bayes' Theorem. It assumes that the input features are conditionally independent of each other given the class.

The word “**Naïve**” is used because the algorithm makes a simplifying independence assumption about features.

Bayes' Theorem

Naïve Bayes is based on the following formula:

$$P(C|X) = \frac{P(X|C) \times P(C)}{P(X)}$$

Where:

- $P(C|X)$ = Posterior Probability
- $P(X|C)$ = Likelihood
- $P(C)$ = Prior Probability
- $P(X)$ = Evidence

Meaning

The algorithm calculates the probability that a data point belongs to a particular class given its features.

The class having the **highest posterior probability** is selected as the predicted class.

Working of Naïve Bayes

Suppose we want to classify an email as:

Spam or Not Spam

The algorithm examines features such as the occurrence of certain words.

For example:

- "Free"
- "Offer"
- "Winner"
- "Money"

Naïve Bayes calculates:

$$P(\text{Spam}|\text{Features})$$

and

$$P(\text{NotSpam}|\text{Features})$$

If:

$$P(\text{Spam}|\text{Features}) > P(\text{NotSpam}|\text{Features})$$

then the email is classified as **Spam**.

Steps of Naïve Bayes Algorithm

Step 1: Prepare the labelled training dataset.

Step 2: Calculate the prior probability of each class.

Step 3: Calculate the likelihood of the observed features for each class.

Step 4: Apply Bayes' theorem.

Step 5: Calculate posterior probability for every possible class.

Step 6: Select the class having the highest posterior probability.

Flow:

Training Data → Calculate Prior Probabilities → Calculate Likelihood → Apply Bayes' Theorem → Compare Posterior Probabilities → Predicted Class

Why is it Called "Naïve" Bayes?

It is called **Naïve** because it assumes that the features are conditionally independent of each other when the class is known.

For example, while classifying a fruit, features may include:

- Color
- Shape
- Weight

Naïve Bayes treats these features as conditionally independent for its probability calculation, even though they may sometimes be related in real-world data.

This assumption simplifies computation and often gives good results in many classification problems.

Types of Naïve Bayes

There are three common types:

1. Gaussian Naïve Bayes

Used mainly when features are **continuous numerical values** and are modeled using a Gaussian/normal distribution within each class.

2. Multinomial Naïve Bayes

Frequently used for **text classification** where features represent counts or frequencies, such as word counts.

Example:

Spam email classification

3. Bernoulli Naïve Bayes

Used primarily when features are **binary**, such as:

- Yes / No
- Present / Absent
- 1 / 0

Advantages of Naïve Bayes

1. Simple and easy to implement.
2. Fast in training and prediction.
3. Works well with high-dimensional data.
4. Particularly useful for many text-classification problems.
5. Requires relatively less training data in many situations.
6. Can handle multiple classes.

Limitations of Naïve Bayes

1. Its independence assumption may not be true for real-world features.
2. Performance may decrease when features are strongly dependent.
3. A probability estimate can become zero for an unseen feature-class combination unless techniques such as **Laplace smoothing** are used.
4. Its probability estimates are not always well calibrated, even when classification accuracy is good.

Applications of Naïve Bayes

Important applications include:

- Spam detection
- Sentiment analysis
- Document classification
- News classification
- Medical diagnosis
- Recommendation systems
- Language-related classification tasks

Naïve Bayes Example – Customer Purchase Prediction

The referenced Naïve Bayes material uses a shopping example with features such as **Day**, **Discount** and **Free Delivery** to predict whether a customer will buy a product.

Suppose an online store has the following historical data:

Day	Discount	Free Delivery	Buy
Weekday	Yes	Yes	Yes
Weekday	No	No	No
Holiday	Yes	Yes	Yes
Weekend	Yes	No	Yes

Holiday	No	Yes	Yes
Weekend	No	No	No
Holiday	Yes	No	Yes
Weekday	Yes	No	Yes
Weekend	No	Yes	No
Holiday	No	No	No

We want to predict:

Will a customer buy a product when **Day = Holiday, Discount = Yes and Free Delivery = Yes?**

Step 1: Calculate Prior Probabilities

Total records:

10

Number of **Buy = Yes**:

6

Number of **Buy = No**:

4

Therefore:

$$P(Yes) = \frac{6}{10} = 0.6$$

$$P(No) = \frac{4}{10} = 0.4$$

Step 2: Calculate Conditional Probabilities

Among the six **Buy = Yes** records:

Holiday occurs 3 times:

$$P(Holiday|Yes) = \frac{3}{6}$$

Discount = Yes occurs 5 times:

$$P(Discount = Yes|Yes) = \frac{5}{6}$$

Free Delivery = Yes occurs 3 times:

$$P(FreeDelivery = Yes|Yes) = \frac{3}{6}$$

Therefore, the unnormalized score for Buy = Yes is:

$$\text{Score(Yes)} = P(\text{Yes}) \times P(\text{Holiday}|\text{Yes}) \times P(\text{Discount}=\text{Yes}|\text{Yes}) \times P(\text{FreeDelivery}=\text{Yes}|\text{Yes})$$

$$= \frac{6}{10} \times \frac{3}{6} \times \frac{5}{6} \times \frac{3}{6}$$

$$= 0.6 \times 0.5 \times 0.8333 \times 0.5$$

$$\boxed{\text{Score(Yes)} \approx 0.125}$$

Step 3: Calculate Score for Buy = No

There are four **No** records.

Among them:

Holiday occurs 1 time:

$$P(\text{Holiday}|\text{No}) = \frac{1}{4}$$

Discount = Yes occurs 0 times:

$$P(\text{Discount} = \text{Yes}|\text{No}) = 0$$

This creates a **zero-frequency problem**. In a practical Naïve Bayes implementation, **Laplace smoothing** is commonly used to avoid a zero probability.

For a first-year conceptual answer, students can state:

Since the observed training data strongly associates the given combination with purchasing, the classifier favors **Buy = Yes**. In practical implementation, Laplace smoothing should be used for unseen feature-class combinations.

Therefore:

$$\boxed{\text{Predicted Class} = \text{Buy}}$$

K-Means Clustering

K-Means is an unsupervised machine-learning algorithm that divides a dataset into K clusters based on similarity among data points.

Unlike KNN and Naïve Bayes, K-Means does **not require labelled training data**.

Here, **K represents the number of clusters** into which data has to be divided.

What is Clustering?

Clustering is an unsupervised learning technique in which similar data points are grouped together into clusters.

The objective is:

- Data points within the same cluster should be **similar**.
- Data points belonging to different clusters should be **less similar**.

Example

Suppose a shopping company has customer data.

K-Means can divide customers into groups such as:

- Low-spending customers
- Medium-spending customers
- High-spending customers

The groups are discovered from data rather than predefined class labels.

Important Terms in K-Means

Cluster

A group of similar data points.

Centroid

The **mean position** of all data points belonging to a cluster.

K : The number of clusters to be formed.

For example: **K = 3**

means the dataset will be divided into **3 clusters**.

Working of K-Means Algorithm

K-Means works iteratively.

Step 1: Select K

Choose the number of clusters.

Example: $K=3$

Step 2: Initialize Centroids

Select K initial centroid positions.

Step 3: Calculate Distance

Calculate the distance of every data point from each centroid.

Euclidean distance is commonly used.

Step 4: Assign Data Points

Assign every data point to its nearest centroid.

This forms K clusters.

Step 5: Recalculate Centroids

For each cluster, calculate the mean of all its data points.

This mean becomes the **new centroid**.

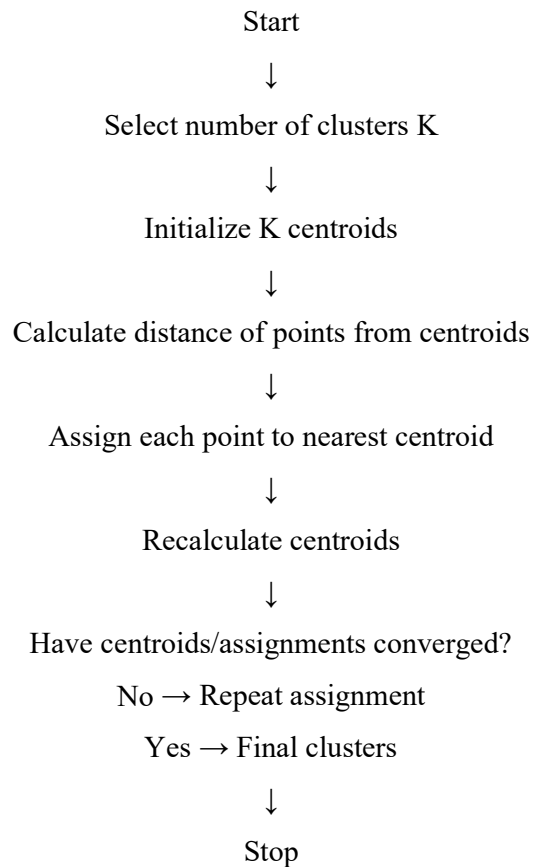
Step 6: Repeat

Again: **Assign points** → **Recalculate centroids**

The process continues until the cluster assignments/centroids stabilize according to the stopping criterion.

K-Means Algorithm Flow

A useful exam diagram is:



K-means Algorithm Example:

Clustering is an unsupervised learning technique in which similar data points are grouped together into clusters.

For example, consider:

10, 12, 15, 70, 75, 80

These data points can be grouped as:

$$Cluster_1 = \{10, 12, 15\}$$

and

$$Cluster_2 = \{70, 75, 80\}$$

The first cluster contains smaller values, whereas the second cluster contains larger values.

What is Centroid:

A centroid is the mean or average of all data points belonging to a cluster.

For example, suppose:

$$Cluster_1 = \{2, 4, 6\}$$

Then its centroid is:

$$C_1 = \frac{2 + 4 + 6}{3}$$

$$C_1 = \frac{12}{3}$$

$$C_1 = 4$$

Steps of K-Means Algorithm

Step 1: Choose the number of clusters, K.

Step 2: Select K initial centroids.

Step 3: Calculate the distance of every data point from each centroid.

Step 4: Assign each data point to the nearest centroid.

Step 5: Calculate new centroids by taking the mean of the data points in each cluster.

Step 6: Again calculate the distances using the new centroids.

Step 7: Reassign the data points to their nearest centroid.

Step 8: Repeat the process until the cluster assignments do not change, or another chosen convergence condition is satisfied.

Distance Formula

For one-dimensional data, the distance between data point x and centroid C is:

$$d = |x - C|$$

For example:

If:

$$x = 10, \quad C = 4$$

then:

$$d = |10 - 4|$$

$$d = 6$$

Consider the following dataset:

$$\{2, 4, 10, 12, 3, 20, 30, 11, 25\}$$

We want to divide these data points into **two clusters**.

Therefore:

$$K = 2$$

Let the initial centroids be:

$$C_1 = 2$$

and

$$C_2 = 4$$

Numerical Example of K-Means

Iteration 1

Initial centroids:

$$C_1 = 2, \quad C_2 = 4$$

Now calculate the distance of every data point from both centroids.

For example, for data point 10:

Distance from C_1 :

$$|10 - 2| = 8$$

Distance from C_2 :

$$|10 - 4| = 6$$

Since 6 is smaller than 8, data point 10 is assigned to Cluster 2.

Distance Table – Iteration 1

Data Point	Distance from C1 = 2	Distance from C2 = 4	Assigned Cluster	
2	0	2	Cluster 1	
4	2	0	Cluster 2	
10	8	6	Cluster 2	
12	10	8	Cluster 2	
3	1	1	Cluster 1*	
20	18	16	Cluster 2	
30	28	26	Cluster 2	
11	9	7	Cluster 2	
25	23	21	Cluster 2	

Iteration 2

Calculate New Centroids

For Cluster 1:

$$C_1 = \frac{2 + 4 + 3}{3}$$

$$C_1 = \frac{9}{3}$$

$$\boxed{C_1 = 3}$$

For Cluster 2:

$$C_2 = \frac{10 + 12 + 20 + 30 + 11 + 25}{6}$$

$$C_2 = \frac{108}{6}$$

$$\boxed{C_2 = 18}$$

Therefore:

$$\boxed{C_1 = 3, \quad C_2 = 18}$$

Again, the centroids have changed. Therefore, continue.

Iteration 4

Now:

$$C_1 = 4.75, \quad C_2 = 19.6$$

Distance Table – Iteration 4

Data Point	Distance from C1 = 4.75	Distance from C2 = 19.6	Assigned Cluster
2	2.75	17.60	Cluster 1
4	0.75	15.60	Cluster 1
10	5.25	9.60	Cluster 1
12	7.25	7.60	Cluster 1
3	1.75	16.60	Cluster 1
20	15.25	0.40	Cluster 2
30	25.25	10.40	Cluster 2
11	6.25	8.60	Cluster 1
25	20.25	5.40	Cluster 2

Therefore:

$$\boxed{Cluster_1 = \{2, 4, 10, 12, 3, 11\}}$$

$$\boxed{Cluster_2 = \{20, 30, 25\}}$$

Notice that 11 and 12 are now assigned to Cluster 1.

Notice that 11 and 12 are now assigned to Cluster 1.

Calculate New Centroids

For Cluster 1:

$$C_1 = \frac{2 + 4 + 10 + 12 + 3 + 11}{6}$$

$$C_1 = \frac{42}{6}$$

$$\boxed{C_1 = 7}$$

For Cluster 2:

$$C_2 = \frac{20 + 30 + 25}{3}$$

$$C_2 = \frac{75}{3}$$

$$\boxed{C_2 = 25}$$

Therefore:

$$\boxed{C_1 = 7, \quad C_2 = 25}$$

Iteration 5 – Final Check

Now:

$$C_1 = 7, \quad C_2 = 25$$

Again calculate all distances.

Distance Table – Iteration 5

Data Point	Distance from $C_1 = 7$	Distance from $C_2 = 25$	Assigned Cluster
2	5	23	Cluster 1
4	3	21	Cluster 1
10	3	15	Cluster 1
12	5	13	Cluster 1
3	4	22	Cluster 1
20	13	5	Cluster 2
30	23	5	Cluster 2
11	4	14	Cluster 1
25	18	0	Cluster 2

Therefore:

$$Cluster_1 = \{2, 4, 10, 12, 3, 11\}$$

$$Cluster_2 = \{20, 30, 25\}$$

These are exactly the same clusters as the previous iteration.

Therefore, there is no change in cluster assignment.

Hence:

K-Means has converged.

7. Final Result

The final clusters are:

$$Cluster_1 = \{2, 3, 4, 10, 11, 12\}$$

with centroid:

$$C_1 = 7$$

and

$$Cluster_2 = \{20, 25, 30\}$$

with centroid:

$$C_2 = 25$$

8. Summary of Centroid Changes

Stage	Centroid C1	Centroid C2
Initial	2	4
After Iteration 1	2.5	16
After Iteration 2	3	18
After Iteration 3	4.75	19.6
After Iteration 4	7	25
Final Check	7	25

Thus, the centroid movement is:

$$\begin{array}{c} (2, 4) \\ \downarrow \\ (2.5, 16) \\ \downarrow \\ (3, 18) \\ \downarrow \\ (4.75, 19.6) \\ \downarrow \\ (7, 25) \end{array}$$

Advantages of K-Means

1. Simple and easy to understand.
2. Easy to implement.
3. Computationally efficient for many datasets.
4. Useful for discovering groups in unlabelled data.
5. Can work well when clusters are compact and reasonably well separated.
6. Useful for customer and data segmentation.

Limitations of K-Means

1. The value of K must be selected in advance.
2. Results can depend on initial centroid selection.
3. It is sensitive to outliers.
4. It works poorly for some non-spherical or highly irregular cluster structures.
5. It is sensitive to differences in feature scales.
6. Different initializations can produce different local solutions.

Applications of K-Means

K-Means is commonly used for:

- Customer segmentation
- Document clustering
- Image segmentation
- Market segmentation
- Grouping similar products
- Pattern analysis
- Data compression / color quantization
- Exploratory analysis of datasets

KNN vs K-Means

Basis	KNN	K-Means
Full Form	K-Nearest Neighbors	K-Means Clustering
Learning Type	Supervised	Unsupervised
Main Task	Classification/Regression	Clustering
Labelled Data	Required for classification	Not required
Meaning of K	Number of nearest neighbors	Number of clusters
Basic Principle	Majority voting / neighbor values	Nearest centroid
Training	No conventional model-fitting phase; stores training examples	Iteratively learns centroids
Output	Predicted class/value	Cluster assignment
Example	Spam/Not Spam classification	Customer segmentation

KNN vs Naïve Bayes

Basis	KNN	Naïve Bayes
Type	Supervised	Supervised
Approach	Distance-based	Probability-based
Main Principle	Nearest neighbors	Bayes' theorem
Training	Stores training data; often called a lazy learner	Estimates class/feature probabilities
Prediction	Can be relatively expensive on large training sets	Usually fast
Key Requirement	Suitable distance metric and scaling	Probabilistic feature model / independence assumption
Common Use	Pattern recognition	Text and document classification

Classification vs Clustering

Classification

Classification is a **supervised learning** technique.

The categories are already known in the training data.

Example: **Student** → **Pass or Fail**

Clustering

Clustering is an **unsupervised learning** technique.

Categories are not provided beforehand. The algorithm discovers groups based on similarity.

Example: **Group students according to similar study/performance patterns.**

Key Difference

Classification predicts predefined class labels, whereas clustering discovers natural groups in unlabelled data.

Classification Metrics — Accuracy, Precision, Recall, F1 Score and MAE

Classification is a **supervised machine learning** task in which a model predicts a **class or category**. Examples include Spam/Not Spam, Disease/No Disease, and Fraud/Not Fraud.

To check how well a classification model performs, we use **evaluation metrics** such as **Accuracy, Precision, Recall, and F1 Score**. The syllabus also mentions **MAE (Mean Absolute Error)**, which is mainly used for numerical prediction/regression.

1. Confusion Matrix

Before studying the metrics, we should understand the **Confusion Matrix**. For binary classification, predictions can be divided into four cases:

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

TP — True Positive: Model predicts Positive and the actual result is also Positive.

Example: A patient has a disease and AI correctly predicts **Disease**.

TN — True Negative: Model predicts Negative and the actual result is also Negative.

Example: A patient does not have the disease and AI correctly predicts **No Disease**.

FP — False Positive: Model predicts Positive, but the actual result is Negative. It is also called a **False Alarm**.

Example: A healthy patient is incorrectly predicted as having the disease.

FN — False Negative: Model predicts Negative, but the actual result is Positive.

Example: A patient actually has the disease, but AI predicts **No Disease**.

2. Accuracy

Definition: Accuracy is the ratio of the number of **correct predictions** to the **total number of predictions**.

Formula:

$$\text{Accuracy} = (TP+TN) / (TP+TN+FP+FN)$$

Example

Suppose an AI model makes predictions for **100 students**:

- TP = 40
- TN = 50
- FP = 5
- FN = 5

Therefore,

$$\text{Accuracy} = (40+50) / (40+50+5+5)$$

Accuracy = 90%

Important Point

Accuracy works well when the classes are reasonably **balanced**. It can be misleading for an **imbalanced dataset**.

For example, if 99 out of 100 transactions are normal and only 1 is fraudulent, a model predicting every transaction as "Normal" gets 99% accuracy even though it completely fails to detect fraud.

3. Precision

Definition: Precision tells us, **out of all samples predicted as Positive, how many were actually Positive?**

Formula: $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

Using the previous example:

$\text{TP} = 40$ and $\text{FP} = 5$

$\text{Precision} = 40 / (40 + 5) = 0.889$

Precision $\approx 88.9\%$

Interpretation

High precision means that when the model predicts **Positive**, its positive prediction is usually correct.

Precision is particularly important when **False Positives are costly**.

Example: Spam detection. If a genuine important email is incorrectly classified as spam, it may create a problem. Therefore, precision can be important in such applications.

4. Recall

Recall is also known as **Sensitivity** or **True Positive Rate (TPR)**.

Definition: Recall tells us, **out of all actual Positive samples, how many were correctly identified by the model?**

Formula: $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

Using our example:

$\text{TP} = 40$ and $\text{FN} = 5$

$\text{Recall} = 40 / (40 + 5) = 0.889$

Recall $\approx 88.9\%$

Interpretation

High recall means the model is able to detect most of the actual positive cases.

Recall becomes particularly important when **missing a positive case (False Negative) is costly**.

Example: In disease screening, failing to identify a patient who actually has a disease may be serious. Therefore, high recall is often important.

5. F1 Score

Sometimes we need to consider both **Precision and Recall**. The **F1 Score** combines them into a single metric.

Definition: F1 Score is the **harmonic mean of Precision and Recall**.

Formula:

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Suppose:

Precision = 0.80

Recall = 0.90

Then,

$$F1 = 2 \times (0.80 \times 0.90) / (0.80 + 0.90) = 0.847$$

F1 Score $\approx$ 84.7%

Why F1 Score?

F1 Score is useful when:

- The dataset is **imbalanced**.
- Both **False Positives and False Negatives** matter.
- We want a balance between **Precision and Recall**.

The value of F1 Score ranges from **0 to 1**, where a value closer to **1** indicates better combined precision and recall.

6. MAE — Mean Absolute Error

MAE stands for Mean Absolute Error.

Unlike Accuracy, Precision, Recall and F1 Score, MAE is mainly an **error metric for regression/numerical prediction**, rather than a standard classification metric.

Definition: MAE calculates the **average absolute difference between actual values and predicted values**.

Formula

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Where:

n = Number of observations

y_i = Actual value

$\hat{y}_i$ = Predicted value

| | = Absolute value

Example

Suppose an AI model predicts students' marks:

Student	Actual Marks	Predicted Marks	Absolute Error
A	80	75	5
B	70	72	2
C	60	66	6
D	90	87	3

Therefore,

$$MAE = (5+2+6+3) / 4$$

MAE = 4 marks

This means that the model's predictions differ from the actual marks by **4 marks on average**.

Important Point

For MAE, a **smaller value is better**:

$$MAE=0$$

means that predictions are exactly equal to actual values for all observations.

Quick Comparison

Metric	Formula	Main Question	Generally
Accuracy	$(TP+TN) / (TP+TN+FP+FN)$	How many total predictions are correct?	Higher is better
Precision	$TP / (TP+FP)$	Of predicted positives, how many are correct?	Higher is better
Recall	$TP / (TP+FN)$	Of actual positives, how many did we find?	Higher is better
F1 Score	$2(P \times R) / (P+R)$	How well are precision and recall balanced?	Higher is better

What is a Neural Network?

An **Artificial Neural Network (ANN)** is a machine-learning model inspired by the working of the **human brain**.

The human brain contains biological neurons that receive and process information. Similarly, an artificial neural network consists of **artificial neurons (nodes)** connected to each other.

A neural network learns patterns from data and uses these patterns to make **predictions or classifications**.

Examples:

- Face recognition
- Handwriting recognition
- Medical image analysis
- Speech recognition
- Spam email detection

Definition

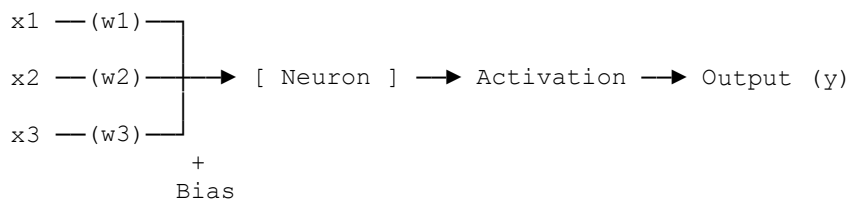
A Neural Network is a computational model consisting of interconnected artificial neurons arranged in layers, which learns patterns from data by adjusting the weights of connections.

Artificial Neuron

An **artificial neuron** is the basic processing unit of a neural network.

It receives one or more inputs, multiplies each input by its corresponding **weight**, adds a **bias**, and passes the result through an **activation function** to generate an output.

Simple Diagram of a Neuron



Where:

x₁, x₂, x₃ = Inputs

w₁, w₂, w₃ = Weights

b = Bias

y = Output

Mathematically:

$$Z = (x_1 * w_1) + (x_2 * w_2) + (x_3 * w_3) + b$$

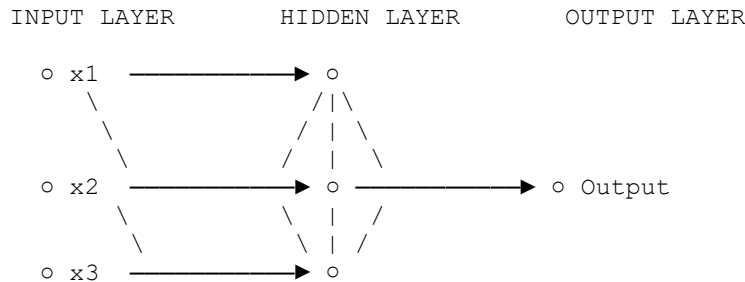
Then,

$$y=f(z)$$

where **f** is the activation function.

Layers in Neural Network

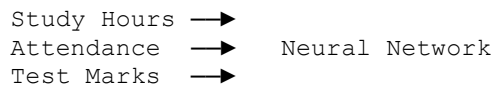
Neurons in a neural network are organized into **layers**. A basic neural network generally contains three types of layers:



(A) Input Layer

The **input layer** receives the initial data.

For example, if a neural network predicts whether a student will pass or fail, inputs could be:



The input layer normally does not perform complex calculations. It passes input values to the next layer.

(B) Hidden Layer

The **hidden layer** lies between the input and output layers.

Its neurons perform calculations and learn important **patterns or features** from the input data.

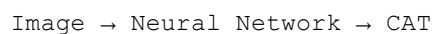
A neural network can have **one or multiple hidden layers**.

More hidden layers allow the network to learn more complex representations. Networks with many hidden layers are commonly called **deep neural networks**.

(C) Output Layer

The **output layer** produces the final result or prediction.

For example:

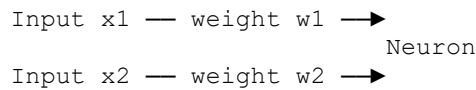


Here, **CAT** is the predicted output.

Weights in Neural Networks

A **weight** is a numerical value associated with a connection between neurons.

Weights represent the **strength or importance of an input/connection**.



A larger positive weight can make an input have a stronger positive influence on the neuron, while a negative weight can push the neuron's calculation in the opposite direction.

During **training**, the neural network adjusts its weights to reduce prediction error.

Therefore:

Learning in a neural network mainly involves adjusting weights and biases so that the network produces better predictions.

Bias:

Bias is an additional adjustable value added to the weighted sum of inputs.

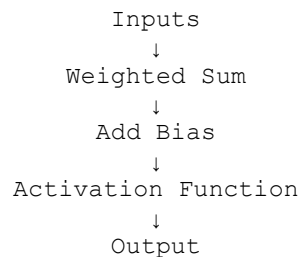
For example:

$$z = (x1 * w1 + x2 * w2) + b$$

Bias helps a neuron shift its response and gives the neural network more flexibility when learning patterns.

Activation Function

After calculating the weighted sum, the neuron usually applies an **activation function**.



The activation function determines how the neuron responds to its calculated input. It also introduces **non-linearity**, which allows neural networks to learn complex patterns.

Common activation functions include **ReLU, Sigmoid and Tanh**.

Working of Neural Network

The basic working can be remembered as:

Input → Weighted Sum → Bias → Activation → Output

For example, suppose:

$x_1=2$, $x_2=3$

and

$w_1=0.5$, $w_2=0.4$, $b=0.2$

Then:

$$z = (2 \times 0.5) + (3 \times 0.4) + 0.2 = 2.4$$

The value **2.4** is then passed through an activation function to obtain the neuron's output.

Activation Function:

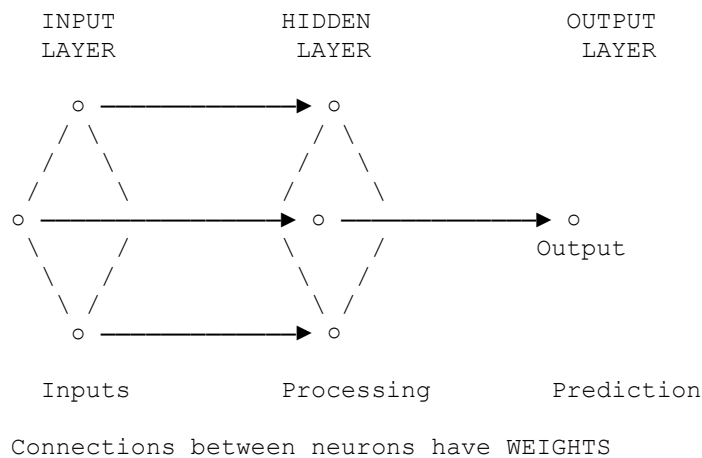
$$\text{Sigmoid}(z) = 1 / (1 + e^{-z})$$

$$\text{Sigmoid}(z) = 1 / (1 + e^{-2.4}) = 0.9168 = 91.68\% \text{ (Probability } \rightarrow \text{ Yes)}$$

During training, if the final prediction is incorrect, the network modifies its **weights and biases** to reduce the error.

Complete Simple Diagram for AKTU Exam

You can draw this diagram in the examination:



Short Note:

Neural Network: An artificial neural network is a computational model inspired by the human brain. It consists of interconnected artificial neurons arranged in layers.

The **input layer** receives data, **hidden layers** process the data and learn patterns, and the **output layer** generates the final prediction. Connections between neurons have numerical values called **weights**, which determine the influence of connections. Each neuron calculates the weighted sum of its inputs, adds a **bias**, and applies an **activation function**.

The basic neuron operation is:

$$y = f(\sum X_i * W_i + b)$$

During training, the network adjusts its weights and biases to reduce prediction error. Neural networks are widely used in **image recognition, speech recognition, NLP, medical imaging and classification problems**.

What is Deep Learning?

Deep Learning (DL) is a branch of **Machine Learning** that uses **neural networks with multiple hidden layers** to learn complex patterns from large amounts of data.

It is called “**deep**” because the neural network contains **many layers of neurons between the input and output**.

Exam Definition

Deep Learning is a subset of Machine Learning that uses artificial neural networks with multiple hidden layers to automatically learn complex features and patterns from data.

Deep learning is commonly used in:

- Image and face recognition
- Speech recognition
- Natural Language Processing (NLP)
- Medical image analysis
- Autonomous vehicles

What Makes Deep Learning “Deep”?

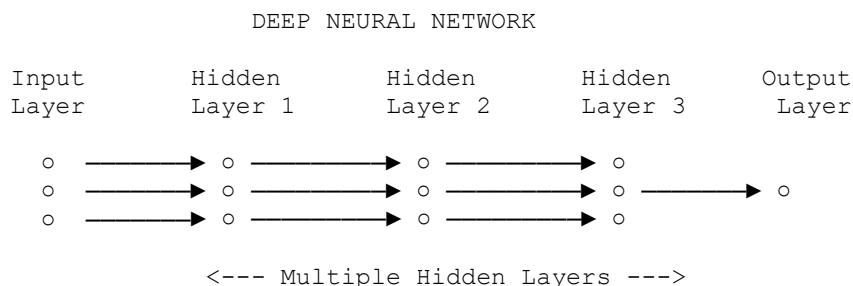
The word **deep** refers to the **depth or number of layers** in a neural network.

Consider a simple neural network:

Input Layer → Hidden Layer → Output Layer

It has only one hidden layer.

A deep neural network contains **multiple hidden layers**:



Therefore:

More hidden layers → greater network depth → Deep Neural Network

However, “deep” does not simply mean “better.” The important idea is that multiple layers can learn **hierarchical representations** of complex data.

How Does Deep Learning Learn Features?

An important advantage of deep learning is **automatic feature learning**.

Suppose a deep learning model is given an image of a human face.

Different layers may progressively learn:

```
Input Image
  ↓
Layer 1 → Edges and lines
  ↓
Layer 2 → Shapes and textures
  ↓
Layer 3 → Eyes, nose, mouth
  ↓
Layer 4 → Complete face
  ↓
Output → Person/Face recognized
```

Thus, earlier layers generally learn **simple features**, while deeper layers can combine them into **more complex features**.

This hierarchical feature learning is one of the main reasons deep learning is effective for images, audio, text and other complex data.

Machine Learning vs Deep Learning

Machine Learning	Deep Learning
Broad family of learning methods	Subfield of machine learning
Can use models such as decision trees, SVMs, etc.	Primarily uses multi-layer neural networks
Often relies more on manually designed features	Can automatically learn useful features
Can work well with smaller datasets depending on task	Often benefits substantially from large datasets
Usually requires less computation	Training can require very high computation
Often runs efficiently on CPUs	Frequently accelerated using GPUs

Why Does Deep Learning Use GPUs?

GPU stands for **Graphics Processing Unit**.

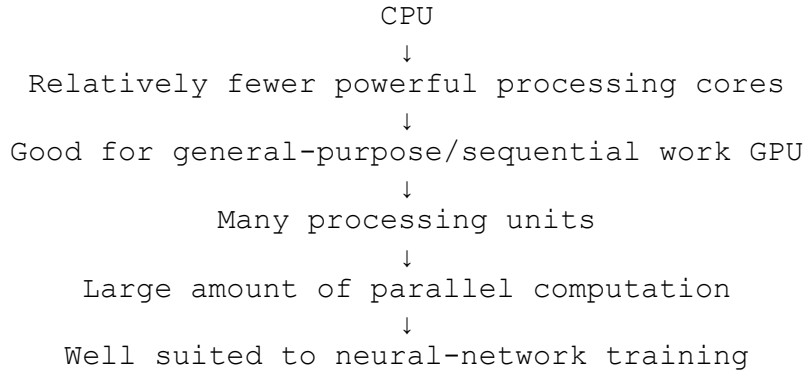
Originally GPUs were designed for graphics processing, but their architecture is also highly suitable for many deep-learning computations.

Main Reason

Deep neural networks perform a very large number of numerical operations, particularly **matrix and tensor operations**.

For example, a network may contain millions or billions of parameters that must repeatedly be used and updated during training.

GPUs can execute many suitable calculations **in parallel**.



Why GPUs are Useful in Deep Learning

- 1. Parallel Processing:** GPUs can perform many mathematical operations simultaneously.
- 2. Faster Matrix Operations:** Neural networks rely heavily on matrix multiplication, and GPUs are optimized for highly parallel numerical workloads.
- 3. Faster Training:** Large deep-learning models can take a long time to train. GPUs can significantly reduce training time compared with CPU-only computation for suitable workloads.
- 4. Large Datasets:** Deep-learning applications often process large quantities of images, audio, text or other data.
- 5. Large Neural Networks:** Modern networks may contain millions or billions of parameters, requiring substantial computational power.

GPUs are used in deep learning because neural networks require a large number of parallel mathematical operations, especially matrix and tensor calculations. GPUs provide high parallel processing capability and therefore can significantly speed up the training of deep neural networks.

What is CNN (Convolutional Neural Network)?

CNN stands for Convolutional Neural Network.

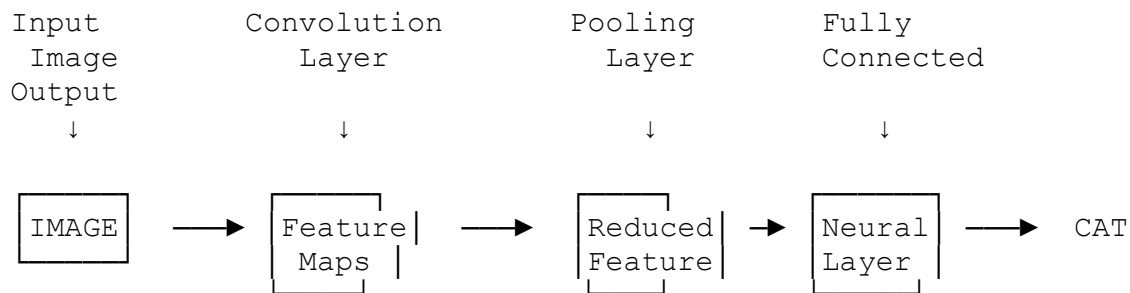
A CNN is a type of neural network especially useful for processing **images and other grid-like data**.

It automatically learns visual features such as:

Edges → Shapes → Object parts → Objects

A Convolutional Neural Network (CNN) is a deep-learning neural network designed to automatically extract spatial features from images and perform tasks such as image classification and object recognition.

Simple CNN Diagram



Main Components of CNN

1. Convolution Layer

The **convolution layer** extracts important features from an image using small learnable filters or kernels.

For example, it can learn patterns useful for detecting:

- Edges
- Curves
- Textures
- Shapes

Image → Convolution/Filters → Feature Maps

It is one of the most important components of a CNN.

2. Activation Function

An activation function such as **ReLU** is commonly applied to introduce non-linearity.

Convolution → ReLU → Feature Map

3. Pooling Layer

A **pooling layer** reduces the spatial dimensions of feature maps.

This can:

- Reduce computation
- Reduce the amount of intermediate data
- Retain important information

A common method is **Max Pooling**.

4. Fully Connected Layer

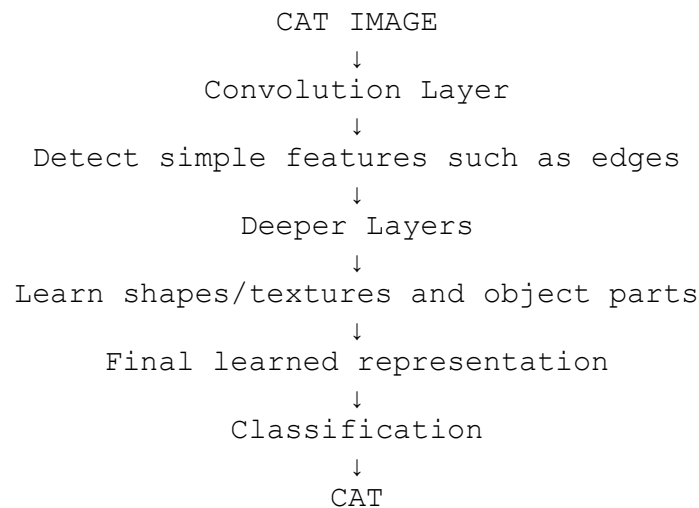
Near the end of a typical CNN, learned features can be passed to **fully connected layers** or another prediction head to produce the final result.

Example:

Learned Features → Classification → Dog

How CNN Recognizes an Image

Suppose we provide an image of a cat:



The network learns useful filters during training rather than requiring a programmer to manually specify every visual feature.

Applications of CNN

CNNs are widely used in **computer vision**, including:

- Face recognition
- Image classification
- Object detection
- Medical image analysis
- Handwriting recognition
- Visual inspection systems
- Autonomous-driving perception

Deep Learning — What Makes It Deep, Why GPUs and CNN

Deep Learning is a subset of machine learning that uses neural networks containing multiple hidden layers. The term “**deep**” refers to the presence of multiple processing layers. These layers can learn hierarchical features, from simple patterns to more complex representations.

Deep-learning models require large numbers of mathematical operations during training.

GPUs (Graphics Processing Units) are useful because they can perform many calculations in parallel and efficiently accelerate matrix and tensor operations. Hence, GPUs can greatly reduce neural-network training time.

A **CNN (Convolutional Neural Network)** is a deep-learning architecture commonly used for image processing. Its convolution layers learn spatial features such as edges, textures and shapes. Pooling can reduce feature-map size, and later layers use the learned features to generate predictions.

Example of CNN to detect horizontal and vertical lines in an image:

1. Input image — 5×5

```
0    0    0    0    0
0    0    0    0    0
0    0    0    0    0
255  255  255  255  255
255  255  255  255  255
```

Here:

- 0 = black
- 255 = white

So there is a **horizontal boundary** between the 3rd and 4th rows.

2. Horizontal edge detection filter

Let's take this 3×3 filter:

```
-1   -1   -1
 0    0    0
 1    1    1
```

This filter is designed to detect a **horizontal edge**.

3. First position

The filter looks at the top-left 3×3 area:

Image		Filter				
0	0	0		-1	-1	-1
0	0	0	×	0	0	0
0	0	0		1	1	1

Multiply corresponding values:

$$\begin{aligned} & (0 \times -1) + (0 \times -1) + (0 \times -1) \\ & + (0 \times 0) + (0 \times 0) + (0 \times 0) \\ & + (0 \times 1) + (0 \times 1) + (0 \times 1) \\ & = 0 \end{aligned}$$

So first output = 0.

4. Move the filter down by one row

Now consider:

```

0    0    0
0    0    0
255  255  255

```

Apply the filter:

$$\begin{aligned}
 &(0 \times -1) + (0 \times -1) + (0 \times -1) \\
 &+ (0 \times 0) + (0 \times 0) + (0 \times 0) \\
 &+ (255 \times 1) + (255 \times 1) + (255 \times 1) \\
 &= 255 + 255 + 255 \\
 &= 765
 \end{aligned}$$

So we get **765**.

This is important: **765 is a strong response from the filter**, meaning a horizontal edge has been detected at this position.

5. Continue sliding

Because the image is 5×5 and filter is 3×3 , with **stride = 1** and **padding = 0**:

$$\begin{aligned}
 \text{Output size} &= (5 - 3) / 1 + 1 \\
 &= 3 \times 3
 \end{aligned}$$

The complete feature map will be:

```

0    0    0
765  765  765
765  765  765

```

Why?

The filter is basically asking:

"Is there a change from dark pixels to bright pixels across these rows?"

At the boundary:

```

0 0 0
0 0 0
255 255 255

```

the answer is strongly **YES**, giving:

```
765
```

That's how a CNN's **convolution layer detects a horizontal edge**.

Exam-friendly flow:

```

5x5 Input Image
    ↓
3x3 Horizontal Edge Filter
    ↓
Multiply + Add
    ↓
Filter slides across image
    ↓

```

3×3 Feature Map
↓
Large value (765) = Horizontal edge detected

Assignment:

For **vertical line / vertical edge detection**, use a filter that compares the **left side and right side** of the image.

Take this **5×5 image**:

0	0	0	255	255
0	0	0	255	255
0	0	0	255	255
0	0	0	255	255
0	0	0	255	255

Here the first 3 columns are black (0) and the last 2 columns are white (255). So there is a **vertical edge** between column 3 and column 4.

Use this **3×3 vertical edge filter**:

-1	0	1
-1	0	1
-1	0	1

Introduction to Computer Vision

Computer Vision is a branch of Artificial Intelligence (AI) that enables computers to understand and interpret information from images and videos.

Humans can look at an image and easily identify objects, faces, colors, shapes, or activities. A computer, however, receives an image as **numerical pixel values**. Computer Vision techniques process these pixels and extract meaningful information from them.

Example

If we provide an image of a car to a Computer Vision system:

Image
↓
Pixel Information
↓
Feature Extraction
↓
AI Model
↓
"Car"

Thus, Computer Vision converts **visual information into meaningful information that a computer can process**.

How Does AI See an Image?

A computer does not see an image in the same way humans do.

An image is represented as a collection of **pixels**.

Grayscale Image

In a grayscale image, each pixel usually has a value between:

0 → Black
255 → White

For example:

0	0	0
100	150	200
255	255	255

These numbers represent the intensity or brightness of the pixels.

Color Image

A color image is commonly represented using three channels:

- **R – Red**
- **G – Green**
- **B – Blue**

Therefore, a color image can be represented as:

Image = Red Channel + Green Channel + Blue Channel

Each pixel contains values corresponding to these channels.

How Does AI Understand an Image?

AI does not directly understand an image as "this is a person" or "this is a car."

It learns **patterns and features** from the image.

The understanding happens progressively:

```
Pixels
  ↓
Edges
  ↓
Lines and Curves
  ↓
Shapes
  ↓
Object Features
  ↓
Object / Face / Region
```

For example, while recognizing a face, the system may learn patterns related to:

- Eyes
- Nose
- Mouth
- Face shape
- Relative positions of facial features

The deep learning model combines these features to produce the final result.

Note: Since CNN has already been covered, we can consider CNN here as one of the important models used for extracting visual features rather than repeating its architecture.

Major Tasks in Computer Vision

Computer Vision is used for different types of tasks. The three important tasks are:

Image Classification

Classification answers:

"What is present in this image?"

For example:

```
Input Image
    ↓
AI Model
    ↓
Cat
```

If the model is trained for multiple classes, the output could be:

```
Cat      → 90%
Dog       → 7%
Other    → 3%
```

The system assigns the image to a particular class.

Applications

- Animal classification
- Plant identification
- Product classification
- Medical image classification

Object Detection

Object Detection goes one step further than classification.

It answers two questions:

1. **What object is present?**
2. **Where is the object located?**

For example, consider a road image containing cars and people.

```
Road Image
    ↓
Object Detection
    ↓
Car → Location
Person → Location
```

The location is commonly represented using a **bounding box** around the detected object.

Example

A traffic monitoring system may detect:

```
Car      → Bounding Box
Person   → Bounding Box
Bus      → Bounding Box
```

Applications

- Traffic monitoring
- Autonomous driving
- Security systems
- Surveillance
- Industrial inspection

Image Segmentation

Image Segmentation means dividing an image into meaningful regions or assigning labels to pixels.

In simple words:

Segmentation tells us which pixels belong to which object or region.

For example, in a medical image, the system may separate:

```
Background
      +
Organ
      +
Region of Interest
```

Segmentation provides more detailed information than object detection because it can identify the **exact region/pixels** associated with an object.

Applications

- Medical imaging
- Satellite image analysis
- Autonomous vehicles
- Background removal
- Industrial inspection

Face Detection and Face Recognition

Face-related applications are among the most common applications of Computer Vision.

However, **Face Detection and Face Recognition are not the same thing.**

Face Detection

Face Detection answers:

"Where is a face in the image?"

For example:

```
Image
↓
Face Detection
↓
Face found
↓
Bounding Box
```

The system locates the face in an image or video.

Face Recognition

Face Recognition answers:

"Whose face is this?"

It compares the facial representation of a detected face with previously enrolled identities.

Face Recognition – Working

A simplified face recognition system works in the following steps:

Step 1: Image Capture

A camera captures the person's face.

Step 2: Face Detection

The system detects the location of the face.

Step 3: Feature Extraction

The system extracts useful information from the face.

For example:

- Shape of the face
- Position of eyes
- Nose and mouth patterns
- Other learned facial characteristics

Modern deep-learning systems commonly represent these characteristics as a numerical **face embedding**.

Step 4: Comparison

The new face representation is compared with stored representations.

Step 5: Result

The system determines whether the face matches an enrolled identity.

```
Camera
↓
Face Detection
↓
Feature Extraction
↓
Face Representation
↓
Comparison
↓
Match / No Match
```

Real-Life Example: Face Unlock

Consider the face-unlock feature of a smartphone.

When the user looks at the phone:

```
Face
↓
Camera
↓
Face Detection
↓
Feature Extraction
↓
Face Representation
↓
Compare with enrolled representation
↓
Authentication Result
```

If the system determines that the face matches the enrolled user, access can be granted.

Therefore, **Face Recognition is an application of Computer Vision and AI.**

Medical Image Analysis

Computer Vision is increasingly used to assist with the analysis of medical images.

Medical images can include:

- X-rays
- CT scans
- MRI scans
- Ultrasound images
- Retinal images
- Microscopic images

The objective is to extract useful information from these images.

Basic Process

```
Medical Image
↓
Preprocessing
```

↓
Feature Extraction
↓
AI / Deep Learning Model
↓
Analysis / Prediction

Example: X-Ray Image Analysis

Suppose an X-ray image is given to an AI system.

The model has been trained using many labelled X-ray images.

During analysis, it may identify patterns associated with different categories.

For example:

X-Ray Image
↓
Image Processing
↓
Feature Extraction
↓
Trained AI Model
↓
Classification / Detection
↓
Result

The result may indicate that an image belongs to a particular category or that a region requires further clinical review.

Important: AI-based medical image analysis is generally used as a **support tool**. Final diagnosis and clinical decisions should involve qualified healthcare professionals.

Computer Vision in Healthcare

Some applications of Computer Vision in healthcare include:

1. Medical Image Classification

Classifying medical images into predefined categories.

2. Disease/Abnormality Detection

Identifying image patterns that may require further examination.

3. Image Segmentation

Identifying specific anatomical structures or regions.

4. Medical Measurement

Extracting measurements such as size, area, or shape from images.

5. Retinal Image Analysis

Analyzing images of the retina for patterns associated with certain eye conditions.

Face Recognition vs Medical Image Analysis

Both are applications of Computer Vision, but their objectives are different.

Face Recognition	Medical Image Analysis
Works mainly with facial images	Works with medical images
Used for identification or verification	Used for medical image analysis
Extracts facial representations	Extracts medical/image features
Used in authentication and security	Used for clinical decision support
Example: Face Unlock	Example: X-ray analysis

Advantages of Computer Vision

Computer Vision provides several benefits:

- Automates visual inspection
- Can process large numbers of images
- Helps identify patterns
- Can assist human experts
- Can work continuously on video streams
- Useful in healthcare, security, transportation, and manufacturing

Limitations of Computer Vision

Computer Vision systems also have limitations:

1. Data Dependency

AI models generally require suitable training data.

2. Image Quality

Poor lighting, blur, noise, or occlusion can affect performance.

3. Bias in Data

If training data is not sufficiently representative, model performance may vary across different conditions or populations.

4. Computational Requirements

Advanced Computer Vision models can require significant computing resources.

5. Privacy and Security

Applications such as face recognition involve important privacy and data-protection considerations.

Applications of Computer Vision

Computer Vision is used in many fields.

Healthcare

- X-ray analysis
- MRI and CT analysis
- Retinal image analysis
- Medical segmentation

Security

- Face detection
- Face recognition
- Object detection

Transportation

- Lane detection
- Vehicle detection
- Pedestrian detection
- Traffic monitoring

Agriculture

- Crop monitoring
- Plant disease analysis
- Fruit detection

Manufacturing

- Product inspection
- Defect detection
- Quality control

Retail

- Product recognition
- Automated checkout
- Inventory monitoring

Natural Language Processing (NLP)

Natural Language Processing (NLP) is a branch of **Artificial Intelligence (AI)** that enables computers to understand, interpret, process, and generate human language in the form of **text and speech**.

Human language is complex because the same word or sentence can have different meanings depending on the context. NLP uses techniques from **Artificial Intelligence, Machine Learning, Linguistics, and Statistics** to process this language.

Applications of NLP

1. **Chatbots** – ChatGPT, customer-support bots
2. **Machine Translation** – English → Hindi, etc.
3. **Speech Recognition** – converting speech into text
4. **Sentiment Analysis** – identifying positive/negative opinions
5. **Search Engines** – understanding search queries
6. **Spam Detection** – identifying spam emails
7. **Text Summarization**
8. **Question Answering Systems**
9. **Grammar Checking**
10. **Virtual Assistants** – Siri, Alexa, Google Assistant

Basic NLP Processing Pipeline

Before a computer can understand text, the text usually goes through several processing steps.

General NLP Pipeline

Raw Text → Cleaning → Tokenization → Stop-word Removal → Stemming/Lemmatization → Feature Extraction → NLP Model → Output

For example:

“The students are studying NLP concepts.”

After preprocessing, it may become:

Output: students study NLP concepts

The major basic NLP operations are:

- Text Cleaning
- Tokenization
- Stop-word Removal
- Stemming
- Lemmatization
- Part-of-Speech Tagging
- Named Entity Recognition
- Feature Extraction
- Text Classification/Prediction

Text Pre-processing

Text preprocessing means converting raw, unstructured text into a clean and suitable form for further NLP processing.

Common preprocessing operations

- Convert text to lowercase
- Remove unnecessary punctuation
- Remove extra spaces

- Remove special characters
- Tokenization
- Stop-word removal
- Stemming
- Lemmatization

Example

Original sentence:

“The Students are Learning NLP!”

After preprocessing:

“students learning nlp”

Tokenization

Tokenization is the process of breaking text into smaller units called **tokens**.

A token can be:

- Word
- Sentence
- Character
- Subword

Word Tokenization

Consider:

“AI understands human language.”

Tokens:

AI | understands | human | language

Therefore,

Number of tokens = 4

Sentence Tokenization

Consider:

“NLP is useful. It is used in chatbots.”

The sentences are:

1. NLP is useful.
2. It is used in chatbots.

Why is Tokenization Important?

Computers cannot directly process a complete paragraph in the same way humans do. Tokenization converts text into manageable units that NLP algorithms can process.

Tokenization is the process of dividing a text into smaller meaningful units called tokens, such as words, sentences, or subwords.

Stop Words

Stop words are commonly occurring words that often provide little useful information for a particular NLP task.

Examples:

a, an, the, is, are, am, was, were, in, on, of, and, to

Consider:

“The boy is playing in the park.”

After stop-word removal:

“boy playing park”

Here:

- The → removed
- is → removed
- in → removed
- the → removed

Advantages

- Reduces the size of the text
- Reduces computational cost
- Removes some unnecessary information
- Can improve performance in certain NLP tasks

Disadvantage

Stop words should **not always be removed**.

For example:

“This movie is not good.”

If “**not**” is removed:

“movie good”

The meaning changes completely.

Therefore, stop-word removal depends on the application.

Stemming

Stemming is the process of reducing words to their **root or base-like form** by removing prefixes or suffixes, usually using simple rules.

The resulting word may not be a valid dictionary word.

Example

Word	Stem
playing	play
played	play
studies	studi
studying	studi
connected	connect
connection	connect

Notice:

studies → studi

“studi” is not a proper English word.

Advantages

- Simple
- Fast
- Requires less computation
- Useful for search and information retrieval

Disadvantages

- May produce incorrect words
- Can remove too much or too little
- Does not understand the actual meaning of the word

Definition

Stemming is a rule-based technique that reduces words to their stem by removing prefixes or suffixes.

Lemmatization

Lemmatization converts a word into its **dictionary base form**, called a **lemma**, using linguistic information.

Unlike stemming, lemmatization tries to produce a valid word.

Example

Word	Lemma
running	run

ran	run
better	good
studies	study
children	child
mice	mouse

For example:

“The children are running.”

After lemmatization:

“The child be run.”

Depending on the NLP system and grammatical information, the exact output can vary.

Stemming vs Lemmatization

Stemming	Lemmatization
Uses simple rules	Uses linguistic knowledge
Faster	Relatively slower
May produce invalid words	Produces valid dictionary words
Less accurate	More accurate
Example: studies → studi	studies → study

Easy way to remember

Stemming = Cutting

Lemmatization = Understanding

Bag of Words (BoW)

The **Bag of Words (BoW)** model represents text using the frequency of words.

It **ignores grammar and word order** and focuses mainly on whether and how frequently words occur.

Example

Suppose we have two sentences:

Sentence 1:

“I like machine learning.”

Sentence 2:

“I like artificial intelligence.”

First create the vocabulary:

[I, like, machine, learning, artificial, intelligence]

Now represent each sentence as a vector.

Word	I	like	machine	learning	artificial	intelligence
Sentence 1	1	1	1	1	0	0
Sentence 2	1	1	0	0	1	1

So:

Sentence 1 → [1, 1, 1, 1, 0, 0]

Sentence 2 → [1, 1, 0, 0, 1, 1]

The numbers represent how many times each word occurs.

Advantages of BoW

- Simple to understand
- Easy to implement
- Useful for text classification
- Works well with traditional ML algorithms

Disadvantages

1. Ignores word order
2. Does not understand context
3. Creates high-dimensional vectors
4. Produces sparse representations

For example:

“Dog bites man.”

and

“Man bites dog.”

BoW may represent both similarly because they contain the same words, even though their meanings are different.

TF-IDF

TF-IDF stands for:

Term Frequency – Inverse Document Frequency

It is a numerical technique used to determine how **important a word is in a document relative to a collection of documents**.

The main idea is:

A word is important if it occurs frequently in a particular document but does not occur frequently in many other documents.

Term Frequency (TF)

TF measures how frequently a word occurs in a document.

A simple formula is:

$TF(t,d) = \text{Number of times term } t \text{ occurs in document } d / \text{Total number of terms in document } d$

Example

Document:

“NLP is useful and NLP is powerful.”

Total words = 7

“NLP” occurs 2 times.

Therefore:

$TF(NLP) = 2 / 7$

Inverse Document Frequency (IDF)

IDF measures how rare or unique a word is across documents.

A common simplified formula is:

$IDF(t) = \log(N / df(t))$

Where:

- **N** = total number of documents
- **df(t)** = number of documents containing term t

If a word appears in almost every document, its IDF is low.

If a word appears in only a few documents, its IDF is high.

TF-IDF Formula

$TF-IDF = TF \times IDF$

Example Concept

Suppose there are 100 documents.

Word “the” appears in 95 documents.

Word “**blockchain**” appears in only 5 documents.

Then:

- “the” → low IDF
- “blockchain” → high IDF

Therefore, “**blockchain**” gets a **higher TF-IDF value** when it is relevant to a particular document.

Applications

- Search engines
- Document ranking
- Keyword extraction
- Text classification
- Information retrieval

TF-IDF vs BoW

BoW: “How many times does this word occur?”

TF-IDF: “How important is this word compared with the entire document collection?”

Word2Vec

Word2Vec is a technique for representing words as **dense numerical vectors (word embeddings)**.

Unlike BoW, Word2Vec can capture **semantic relationships between words** based on their context.

For example, words such as:

king, queen, man, woman

can have meaningful relationships in vector space.

A simplified conceptual relationship is:

king – man + woman $\approx$ queen

The exact vectors are learned from data; this equation is used to illustrate the semantic structure captured by embeddings.

How Word2Vec Works

Word2Vec learns word representations from the context in which words appear.

There are two major architectures:

1. CBOW – Continuous Bag of Words

CBOW predicts the **target word from surrounding context words**.

Example:

“The cat is ____ on the mat.”

Context:

The, cat, is, on, the, mat

The model tries to predict a suitable word such as:

sitting

2. Skip-gram

Skip-gram does the opposite.

It uses a **target word to predict surrounding context words**.

Example:

“The cat is sitting on the mat.”

If target word is:

“sitting”

The model tries to predict nearby words such as:

cat, is, on, the

CBOW vs Skip-gram

CBOW	Skip-gram
Context → Target word	Target word → Context
Usually faster	Usually slower
Good for frequent words	Can work well with rare words
Predicts missing word	Predicts surrounding words

Word Embeddings

A **word embedding** represents a word as a vector of real numbers.

For example, a simplified representation could be:

king → [0.25, 0.81, -0.12, 0.44, ...]

queen → [0.28, 0.79, -0.10, 0.47, ...]

These numbers are learned from large amounts of text.

Words with similar meanings tend to have similar vector representations.

For example:

king ↔ queen

car ↔ automobile

happy ↔ joyful

This is much more useful than simply representing words as 0 and 1.

Part-of-Speech (POS) Tagging

Part-of-Speech tagging assigns a grammatical category to each word.

Common POS categories:

- Noun
- Verb
- Adjective
- Adverb
- Pronoun
- Preposition
- Conjunction
- Determiner

Example

“Rahul plays cricket.”

Word	POS
Rahul	Noun/Proper Noun
plays	Verb
cricket	Noun

POS tagging helps NLP systems understand the grammatical structure of sentences.

Named Entity Recognition (NER)

Named Entity Recognition identifies important entities in text and classifies them into categories.

Common entities include:

- Person, Organization
- Location, Date, Time
- Money, Product

Example

“Rahul works at Microsoft in Delhi.”

NER may identify:

- Rahul → **PERSON**
- Microsoft → **ORGANIZATION**
- Delhi → **LOCATION**

Applications

- Information extraction
- Search engines
- Question answering
- News analysis
- Chatbots

Syntax Analysis / Parsing

Syntax analysis determines how words are grammatically related in a sentence.

Example:

“The boy eats an apple.”

The system can identify:

- The boy → Subject
- eats → Verb
- an apple → Object

This helps the computer understand the grammatical structure of a sentence.

Semantic Analysis

Semantic analysis deals with the **meaning of words and sentences**.

For example: “I went to the bank to withdraw money.”

The system should understand that **bank** refers to a financial institution.

Semantic analysis helps resolve such meaning-related problems.

Sentiment Analysis

Sentiment analysis determines the emotional or opinion-based polarity of text.

Typical categories:

- Positive
- Negative
- Neutral

Example

“The movie was excellent.”

Output:

Positive

Another example:

“The product quality is very poor.”

Output:

Negative

Applications

- Product reviews
- Social media monitoring
- Customer feedback
- Brand analysis

Speech Recognition

Speech recognition converts human speech into text.

Basic process

Speech → Audio Signal → Feature Extraction → Speech Model → Text

Example:

User says:

“What is the weather today?”

Speech recognition converts it into:

“What is the weather today?”

NLP can then process the text and determine the user's intention.

Natural Language Understanding (NLU)

NLU is the part of NLP concerned with understanding the meaning and intent of human language.

For example:

“Book a flight from Delhi to Mumbai tomorrow.”

NLU can identify:

- Intent → Book flight
- Source → Delhi

- Destination → Mumbai
- Date → Tomorrow

Natural Language Generation (NLG)

NLG is the process of generating human-readable language from structured information or machine-generated results.

Example:

Input:

Temperature = 30°C

NLG can generate:

“The temperature today is 30°C.”

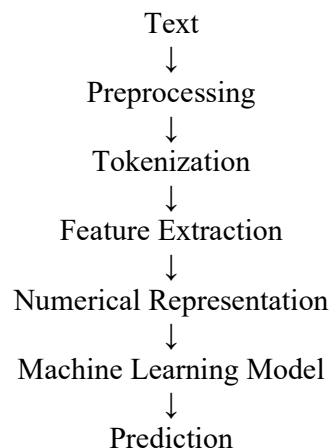
NLG is used in:

- Chatbots
- Virtual assistants
- Automated reports
- AI content generation

NLP in Machine Learning

Traditional NLP systems often convert text into numerical features and then apply machine-learning algorithms.

Example



For sentiment analysis:

“This movie is fantastic.”

could be converted into numerical features and given to a classifier.

The classifier may output:

Positive

Limitations and Challenges of NLP

NLP is difficult because human language is highly complex.

Major challenges

1. Ambiguity

The same word can have different meanings.

“I went to the bank.”

Which type of bank?

2. Context

“This is sick!”

Depending on context, *sick* can mean ill or can be slang for something impressive.

3. Sarcasm

“Great! Another exam tomorrow. Wonderful!”

The literal words are positive, but the actual sentiment is negative.

4. Multiple Languages

Different languages have different grammar, vocabulary, and structures.

5. Spelling Errors

“I luv this movie.”

The system needs to understand that “luv” probably means “love.”

6. Slang and Abbreviations

“OMG, this is awesome!”

7. Speech Variations

Different accents, pronunciation, background noise, and speaking speeds make speech recognition difficult.

Important NLP Concepts at a Glance

Concept	Main Purpose	Example
Tokenization	Split text into tokens	“I love AI” → I, love, AI
Stop-word Removal	Remove common words	“the boy is here” → “boy here”
Stemming	Reduce to stem	studies → studi
Lemmatization	Find dictionary form	studies → study
BoW	Word-frequency representation	word → count
TF-IDF	Measure word importance	rare words get higher weight
Word2Vec	Learn word embeddings	king → vector
POS Tagging	Identify grammar	plays → Verb
NER	Identify entities	Delhi → Location
Sentiment Analysis	Identify opinion	excellent → Positive
NLU	Understand intent/meaning	“Book a ticket” → booking intent
NLG	Generate language	data → natural sentence